FIAP

MBA

AULA 02

RAG e Bases de Conhecimento Corporativas

Enterprise Architecture with Generative AI — MBA Executivo em Arquitetura de Soluções

Professor: Francisco Carlos Junqueira Junior

AGENDA

Agenda da Aula

Bloco 1

- Recap da Aula 01 + A ponte: o problema que RAG resolve
- O que é RAG? Conceito e motivação
- Pipeline RAG: visão end-to-end
- 4 etapas críticas: Document Loading & Chunking, Embeddings, Vector Databases, Retrieval
- Demo ao vivo: Claude Code consultando base de conhecimento via RAG
- Métricas de qualidade de RAG

Bloco 2 — 21:15–23:30

- Taxonomia: 5 tipos de RAG
- Deep-dive em cada tipo com diagrama de arquitetura
- Matriz de decisão: qual RAG usar?
- Trade-offs para cenários corporativos
- Exercício em grupo: seleção de RAG para cenário real
- Conexão com atividades avaliativas e preview da Aula 03

RECAP — AULA 01

O Problema que RAG Resolve

Na aula anterior, identificamos **4 limitações fundamentais** dos LLMs:

Limitação	O que acontece
Alucinação	Modelo inventa dados com confiança total
Context Rot	Qualidade degrada em conversas longas
Overgeneralization	Recomendações genéricas ignoram contexto
Knowledge Cutoff	Modelo não sabe o que aconteceu após treinamento

> *"RAG é o Grounding em escala industrial: em vez de copiar dados manualmente no prompt, um sistema automatizado busca os dados relevantes e os injeta no contexto do LLM."*

CONCEITO

O que é RAG? Conceito e Motivação

Retrieval-Augmented Generation (RAG) é um padrão arquitetural que combina recuperação de informação (retrieval) com geração de texto (LLM) para produzir respostas fundamentadas em dados específicos.

❌ Sem RAG

```
"Qual foi a decisão sobre uso de
Kubernetes no projeto Alpha?"

→ "Baseado em best practices, a
  decisão típica sobre Kubernetes
  envolve considerar escalabilidade,
  complexidade operacional..."

Resposta genérica, sem dados reais.
```

✅ Com RAG

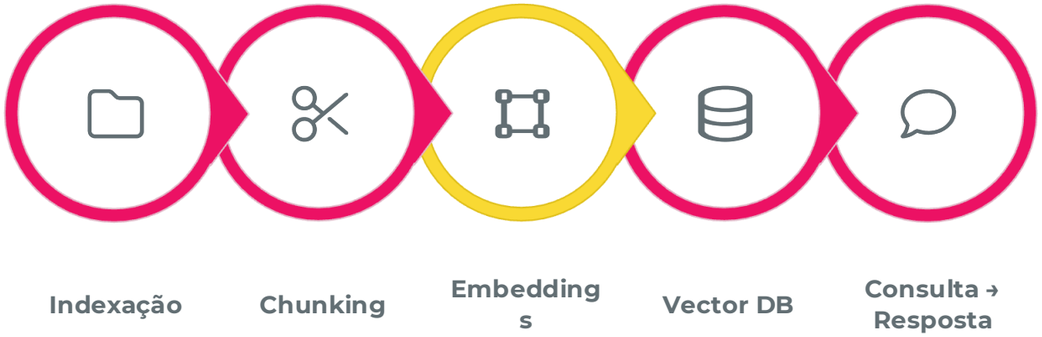
```
"Qual foi a decisão sobre uso de
Kubernetes no projeto Alpha?"

RAG busca: ADR-047, ADR-023,
           Runbook-12

→ "Conforme ADR-047 de março/2024,
  a equipe decidiu adotar Kubernetes
  com as seguintes justificativas:
  [cita dados reais do ADR]..."

Resposta fundamentada em dados reais.
```

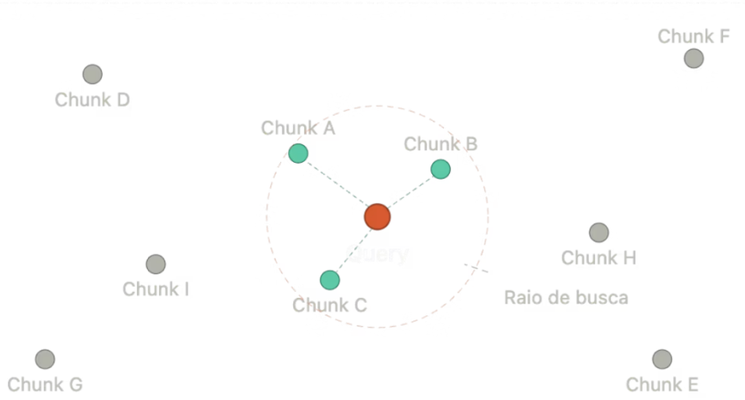
Sem RAG	Com RAG
Apenas dados públicos de treinamento	Acessa dados corporativos atualizados
Respostas genéricas ("best practices")	Respostas contextualizadas ("conforme ADR-047...")
Alto risco de alucinação em dados internos	Grounding automático reduz alucinação
Conhecimento estático (knowledge cutoff)	Conhecimento dinâmico (atualizado em tempo real)

PIPELINE

Pipeline RAG: Diagrama End-to-End

O pipeline RAG conecta documentos corporativos a respostas fundamentadas em dados, passando por etapas críticas de decisão arquitetural.





Cada etapa do pipeline representa uma decisão arquitetural com impacto direto na qualidade e desempenho do sistema.

Etapa	Decisão do Arquiteto	Impacto
Chunking	Como dividir documentos em pedaços (tamanho, sobreposição, estratégia)	Acurácia da busca
Embeddings	Qual modelo vetorial usar (OpenAI, Cohere, modelos open-source)	Qualidade semântica
Vector DB	Onde armazenar e como indexar (Pinecone, Weaviate, pgvector)	Latência e escalabilidade
Retrieval	Como buscar e ranquear resultados (k-NN, MMR, reranking)	Relevância da resposta

PIPELINE

Pipeline RAG - Dois Momentos Distintos: Indexação

O pipeline RAG opera em dois momentos **completamente diferentes** em termos de latência, custo e frequência — cada um com requisitos arquiteturais próprios.

Momento 1 — Indexação (Offline / Batch)

Fluxo: Documentos → Chunking → Embeddings → Vector DB

- Roda em batch, pode levar minutos a horas
- Executado quando novos documentos são adicionados ou atualizados
- Custo principal: processamento de embeddings por documento
- Componentes: Chunking, Embeddings, Vector DB

Momento 2 — Consulta (Online / Real-time)

Fluxo: Query → Embedding da query → Busca no Vector DB → Contexto + Query → LLM → Resposta

- Deve ser rápido: milissegundos a segundos
- Executado a cada pergunta do usuário
- Custo principal: embedding da query + tokens LLM
- Componentes: Retrieval, Prompt Augmentation, LLM

Aspecto	Indexação (Offline)	Consulta (Online)
Quando roda	Batch (diário, semanal ou on-demand)	A cada pergunta do usuário
Latência aceitável	Minutos a horas	Milissegundos a segundos
Custo principal	Embeddings de todos os chunks	Embedding da query + tokens LLM
Frequência	Baixa (atualização de documentos)	Alta (cada interação do usuário)
Componentes	Chunking, Embeddings, Vector DB	Retrieval, Prompt Augmentation, LLM

💡 **Decisão arquitetural crítica:** A separação clara entre indexação e consulta permite otimizar cada momento de forma independente — escalando o pipeline de consulta com caches, rerankers e roteamento inteligente sem impactar o processo de ingestão.

PIPELINE

O fluxo completo

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  DOCUMENTOS  │───>│   CHUNKING   │───>│  EMBEDDINGS  │───>│  VECTOR DB   │
│ corporativos │    │ (dividir em  │    │ (transformar │    │ (armazenar   │
│ (ADRs, wiki, │    │  pedaços)    │    │  em vetores) │    │  e indexar)  │
│  runbooks)   │    │              │    │              │    │              │
└──────────────┘    └──────────────┘    └──────────────┘    └──────┬───────┘
                                                                    │
                                                                    │
                   ┌──────────────┐    ┌──────────────┐           │
                   │  RESPOSTA    │<───│     LLM      │<──────────┘
                   │ fundamentada │    │  (gera com   │    ┌──────────────┐
                   │ em dados     │    │   contexto)  │<───│  RETRIEVAL   │
                   │ corporativos │    │              │    │ (buscar os   │
                   └──────────────┘    └──────────────┘    │  mais        │
                                                           │  relevantes) │
                                                           └──────────────┘
                                          ^
                                          |
                                       ┌──────────┐
                                       │  QUERY   │
                                       │ do usuário │
                                       └──────────┘
```

Etapa	Decisão do arquiteto	Impacto
Chunking	Como dividir documentos em pedaços	Acurácia da busca
Embeddings	Qual modelo vetorial usar	Qualidade semântica
Vector DB	Onde armazenar e como indexar	Latência e escalabilidade
Retrieval	Como buscar e ranquear resultados	Relevância da resposta

ETAPA 1

Document Loading e Chunking

Fontes de Documento

Fonte	Formato	Desafio
Confluence/SharePoint	HTML, markdown	Texto limpo
Repositórios Git	Código, ADRs (.md)	Metadados
PDFs (contratos)	Imagem + texto	OCR, tabelas
ERPs/legados	APIs, SOAP	Adaptadores
Bancos de dados	SQL, NoSQL	Serialização

LLMs têm **context window limitado** (ex: 200K tokens no Claude). Documentos corporativos frequentemente excedem esse limite. Mesmo quando cabem, documentos inteiros diluem a relevância — o LLM precisa encontrar a agulha no palheiro.

4 Estratégias de Chunking

Estratégia	Quando usar
Fixed-size	Docs homogêneos, prototipação
Semantic	Documentos narrativos, relatórios
Recursive	Uso geral — boa default
Document-aware	ADRs, políticas, docs técnicos ✅

💡 **Recomendação:** Para ADRs e runbooks, use **Document-aware chunking** com overlap de 15%.

- Preserva seções lógicas (Decisão, Justificativa, Consequências)
- Mantém metadados (data, autor, status) vinculados ao chunk
- Permite retrieval preciso por seção

```
Chunk 1: [........texto A........]
Chunk 2:              [........texto B........]
                       ^^^^^^^^^^
                       overlap (10-20%) — evita perda de contexto na fronteira
```

ETAPA 2

Embeddings — Transformando Texto em Vetores

Embedding transforma texto em um vetor numérico que captura o *significado semântico*. Textos similares ficam **próximos** no espaço vetorial.

```
"Kubernetes é uma plataforma de orquestração de containers"
        → [0.23, -0.45, 0.87, 0.12, ...]  ─┐
                                           │  PRÓXIMOS (alta similaridade)
"Docker orchestration platform for containers"│
        → [0.21, -0.43, 0.85, 0.14, ...]  ─┘

"A receita de bolo de chocolate leva farinha"
        → [-0.78, 0.34, -0.12, 0.91, ...]  ← DISTANTE (baixa similaridade)
```

Modelo	Provider	Dimensões	Custo	Observação
text-embedding-3-large	OpenAI	3072	$0.13/1M tokens	Estado da arte comercial
text-embedding-3-small	OpenAI	1536	$0.02/1M tokens	Bom custo-benefício ✅
embed-v4	Cohere	1024	$0.10/1M tokens	Suporte multilíngue forte
Voyage-3	Voyage AI	1024	$0.06/1M tokens	Otimizado para código e docs ✅
BGE / E5	Open-source	768–1024	Gratuito (self-hosted)	Sem dependência de provider

🏛 Quando o arquiteto pergunta *"Qual a política de containers?"*, o sistema:

1. Transforma a pergunta em vetor
2. Busca no Vector DB os chunks cujos vetores são mais **próximos**
3. Retorna os chunks sobre containers/Kubernetes — mesmo que a palavra exata "containers" não apareça no chunk (busca **semântica**, não literal)

ETAPA 2

Trade-offs para o arquiteto

Dimensão	Mais dimensões (3072)	Menos dimensões (768)
Qualidade semântica	Mais nuances capturadas	Pode perder sutilezas
Custo de armazenamento	Maior (mais espaço no Vector DB)	Menor
Latência de busca	Mais lento (mais cálculos)	Mais rápido
Custo de API	Mais caro por chamada	Mais barato

Decisão arquitetural

Para a maioria dos cenários corporativos de documentação arquitetural: **text-embedding-3-small** (OpenAI) ou **Voyage-3** são boas defaults. Para cenários com requisitos de privacidade ou sem conexão externa: modelos **open-source (BGE/E5)** self-hosted.

ETAPA 3

Vector Databases — Armazenamento e Busca Semântica

Banco de dados especializado em armazenar, indexar e buscar vetores de alta dimensionalidade por **similaridade** — não por igualdade exata.

Vector DB	Tipo	Hospedagem	Diferencial	Melhor para
Pinecone	Managed SaaS	Cloud only	Simplicidade, zero ops	Times que querem começar rápido
Weaviate	Open-source	Self-hosted/Cloud	Busca híbrida, GraphQL API	Busca híbrida e flexibilidade
Milvus	Open-source	Self-hosted	GPU-accelerated, escala massiva	Grandes volumes, alta performance
pgvector	Extensão Postgres	Onde Postgres rodar	Usa infra existente	Times que já usam Postgres
Neo4j	Graph DB + Vector	Self-hosted/Cloud	Grafos + vetores	Graph RAG, docs com relações

pgvector

"Já temos Postgres e queremos simplicidade"

Pinecone

"Precisamos de escala e simplicidade operacional"

Weaviate / Milvus

"Temos equipe de infra e queremos controle total"

Neo4j

"Nossos documentos têm relacionamentos complexos"

Comparação para arquitetos

Vector DB	Tipo	Hospedagem	Diferencial	Melhor para
Pinecone	Managed SaaS	Cloud only	Simplicidade, escala automática, zero ops	Times que querem começar rápido, sem infra própria
Weaviate	Open-source + Cloud	Self-hosted ou Cloud	Busca híbrida (vetor + keyword), GraphQL API	Cenários que precisam de busca híbrida e flexibilidade
Milvus	Open-source	Self-hosted	Escala massiva (bilhões de vetores), GPU-accelerated	Grandes volumes, alta performance
pgvector	Extensão PostgreSQL	Onde Postgres rodar	Usa infraestrutura existente, sem novo serviço	Times que já usam Postgres e querem simplicidade
Neo4j	Graph DB + Vector	Self-hosted ou Cloud	Combina grafos de conhecimento com busca vetorial	Graph RAG, documentação com relacionamentos complexos

Matriz de decisão

Critério	Pinecone	Weaviate	Milvus	pgvector	Neo4j
Simplicidade	Alta	Média	Baixa	Alta	Média
Escala	Alta	Média	Muito alta	Média	Média
Custo	$$ (SaaS)	$ (self-hosted)	$ (self-hosted)	$ (existente)	$$
Busca híbrida	Sim	Nativo	Sim	Limitado	Nativo (grafo)
Governança	Depende do provider	Total (self-hosted)	Total	Total	Total
Latência P99	<50ms	<100ms	<50ms	~200ms	~150ms

Decisão do arquiteto

A escolha depende do contexto organizacional:

- **"Já temos Postgres e queremos algo simples"** → pgvector
- **"Precisamos de escala e simplicidade operacional"** → Pinecone
- **"Temos equipe de infra e queremos controle total"** → Weaviate ou Milvus
- **"Nossos documentos têm relacionamentos complexos"** → Neo4j

Vector DB é uma decisão de infraestrutura, não de IA. Os mesmos critérios que usamos para escolher qualquer banco de dados se aplicam: custo, ops, governança, escala, latência.

ETAPA 4

Retrieval e Prompt Augmentation

```
1. Query: "Qual a política de backup para sistemas críticos?"
   ↓
2. Embedding da query: [0.34, -0.21, 0.78, ...]
   ↓
3. Busca no Vector DB: "Top-K chunks mais similares"
   ↓
4. Chunks recuperados:
   - Chunk #142: "Política de Backup v3.2 — RPO 1h, RTO 4h..."
   - Chunk #087: "ADR-031: Decisão sobre estratégia de DR para tier-1..."
   - Chunk #203: "Runbook: Procedimento de restore para banco Oracle..."
   ↓
5. Prompt Augmentation:
   [SYSTEM]: Responda APENAS com base nos documentos fornecidos.
             Se não encontrar, diga "não encontrei nos documentos".
   [CONTEXT]: {chunk #142} {chunk #087} {chunk #203}
   [QUERY]:   Qual a política de backup para sistemas críticos?
   ↓
6. LLM gera resposta fundamentada nos chunks
```

Parâmetro	O que controla	Típico	Trade-off
Top-K	Quantos chunks recuperar	3–10	Mais = contexto mas ruído
Similarity threshold	Mínimo de similaridade	0.7–0.85	Alto = preciso mas pode perder chunks
Reranking	Re-ordenar por relevância	Sim/Não	Melhora acurácia, adiciona latência

DEMO AO VIVO

Claude Code Consultando Base de Conhecimento via RAG

Claude Code é uma ferramenta de linha de comando que funciona como um **agente analítico estruturado**.

Demonstração 1 — Sem RAG

```
Prompt:
"Quais foram as decisões do
projeto Phoenix sobre message
brokers?"

Resposta esperada:
→ Resposta genérica sobre
  message brokers, sem dados
  específicos do projeto
  Phoenix (não está nos dados
  de treinamento do LLM).
```

Demonstração 2 — Com RAG

```
Prompt + Context via RAG:
Mesma pergunta, mas com chunks
relevantes injetados
automaticamente.

Resposta esperada:
→ Cita ADRs específicos,
  decisões reais e
  justificativas
  documentadas.
```

Aspecto	Sem RAG	Com RAG
Especificidade	Genérica	Cita documentos reais
Confiança	Alta (mesmo inventando)	Alta (com fontes)
Verificabilidade	Impossível verificar	Citações rastreáveis
Utilidade para decisão	Baixa	Alta

MÉTRICAS

Métricas de Qualidade de RAG

Métrica	O que mede	Como interpretar	Alvo
Precision@K	Dos K chunks, quantos são relevantes?	K=5 e 3 relevantes: 60%	**>70%**
Recall	Dos chunks relevantes, quantos foram recuperados?	4 existem, 3 recuperados: 75%	**>80%**
MRR	O melhor chunk aparece em qual posição?	1º: MRR=1.0 · 3º: MRR=0.33	**>0.7**
Faithfulness	A resposta é fiel aos chunks recuperados?	O LLM inventou algo fora dos chunks?	**>90%**

```
Medir métricas → Identificar gargalo → Ajustar componente → Remedir
     │
     ├── Precision baixa?      → Ajustar chunking
     ├── Recall baixo?         → Ajustar retrieval (top-K, threshold)
     ├── MRR baixo?            → Adicionar reranking
     └── Faithfulness baixa?   → Ajustar prompt augmentation
```

Precision baixa

Sistema recupera lixo; LLM confunde-se com chunks irrelevantes

Recall baixo

Chunks importantes não são encontrados; respostas incompletas

MRR baixo

O melhor chunk não aparece primeiro; respostas diluídas

Faithfulness baixa

LLM alucina mesmo com contexto — ajustar prompt augmentation

CHECKPOINT

1ª Chamada + Checkpoint do Bloco 1

1 **RAG resolve**

Knowledge cutoff e alucinação via grounding automático

2 **Pipeline tem 4 decisões**

Chunking, embeddings, vector DB, retrieval — cada uma impacta qualidade

3 **Métricas são essenciais**

Precision, recall, MRR, faithfulness — RAG precisa de monitoramento

⏸️ **Intervalo:** 21:00–21:15 (15 minutos)

BLOCO 2

Recapitulação e Transição para o Bloco 2

✅ O que vimos no Bloco 1

- **RAG resolve** knowledge cutoff e alucinação via grounding automático
- **Pipeline RAG** tem 4 decisões: chunking, embeddings, vector DB, retrieval
- **Demo ao vivo** mostrou a diferença dramática com e sem RAG
- **Métricas** permitem avaliar e iterar a qualidade do sistema

🔜 O que veremos agora

Existem **5 tipos de RAG** — o pipeline que vimos é o **Basic RAG**, a versão mais simples.

Cada tipo tem uma arquitetura diferente, para cenários diferentes.

> A pergunta central: **qual tipo usar e quando?**

TAXONOMIA

Taxonomia: 5 Tipos de RAG

Tipo	Conceito em uma frase	Complexidade
Basic RAG	Busca vetorial simples → LLM	Baixa
Advanced RAG	Busca + reranking + query expansion	Média
Graph RAG	Busca em grafos de conhecimento	Alta
Agentic RAG	Agente decide onde e como buscar	Muito alta
Hybrid RAG	Combina busca vetorial + estruturada	Alta

⚖️ Cada tipo **adiciona capacidade** ao anterior, mas também **adiciona complexidade e custo**. A escolha é sempre um trade-off.

TIPO 1

Basic RAG: Quando a Simplicidade é Suficiente

✅ Quando usar

- Documentação **homogênea e bem estruturada** (catálogos, FAQs, manuais)
- Volume **pequeno a médio** (<10K documentos)
- Queries **diretas e simples** ("Qual a política de X?")
- **Prototipação** — validar viabilidade antes de investir em RAG avançado

❌ Quando NÃO usar

- Documentação com **relacionamentos complexos** entre entidades
- Queries que exigem **raciocínio** sobre múltiplos documentos
- Cenários com **múltiplas fontes heterogêneas**

Números de referência

<100ms
Latência

~70%
Acurácia

Custo	$ (mais barato)
Implementação	1–2 semanas
Pipeline	Linear, sem loops
Busca	Puramente vetorial

TIPO 2

Advanced RAG: Melhorando a Qualidade de Busca

Técnica	O que faz	Impacto
Query Expansion	Gera variações da query (sinônimos, reformulações)	Melhora recall
Reranking	Re-ordena chunks por relevância real via modelo dedicado	Melhora precision
HyDE	Gera documento hipotético que responderia a query	Melhora qualidade semântica
Filtros de metadados	Filtra por data, autor, tipo antes da busca	Reduz ruído

✅ Quando usar

- Volume **médio a grande** (10K–100K documentos)
- Queries **ambíguas ou técnicas** que precisam de expansão
- **Precision importa** — erros de busca são caros

Números de referência

Latência	100–500ms
Acurácia	~85%
Custo	$$
Implementação	3–6 semanas

TIPO 3

Graph RAG: Conhecimento como Grafo

O que muda

Basic RAG encontra o documento. Graph RAG encontra o documento e tudo que está conectado a ele.

- "Qual a decisão sobre Kubernetes?" → encontra ADR-047
- "Que sistemas dependem?" → navega ADR-047 → Sistema X, Y
- "Quem é responsável?" → navega Sistema X → Equipe A

Ferramentas e Números

- **Neo4j** + LangChain/LlamaIndex
- **Microsoft GraphRAG** (open-source)
- **Weaviate** (combina vetorial + grafo)

Latência	200–800ms
Acurácia	~90%
Custo	$$$
Implementação	2–4 meses

🎯 **Usar quando:** ADRs interconectados, análise de impacto, rastreabilidade, contexto histórico e dependências importam.

TIPO 4

Agentic RAG: Agentes que Decidem Onde Buscar

O que muda

O **agente analisa a query e decide dinamicamente** onde buscar:

- "Qual a política de backup?" → Vector DB
- "Quanto gastamos em cloud no Q3?" → SQL
- "O serviço X está operacional?" → API de monitoramento
- "Quem decidiu migrar para microserviços?" → Knowledge Graph

Riscos e Mitigações

Risco	Mitigação
Roteamento errado	Fallback para busca em todas as fontes
Latência alta	Busca paralela com timeout
Custo	Cache de decisões de roteamento
Governança	Log detalhado de cada decisão

Latência	500–2000ms
Acurácia	~80% (depende do roteamento)
Custo	$$$$
Implementação	3–6 meses

TIPO 5

Hybrid RAG: Combinando Mundos

Quando busca vetorial falha

Cenário	Solução
Códigos específicos "ADR-047"	Keyword (BM25) — match exato
Nomes próprios "Projeto Phoenix"	Keyword — match literal
Números "SLA 99.99%"	Keyword — busca literal

Quando busca keyword falha

Cenário	Solução
"containers" ≠ "containerização"	Vetorial — captura semântica
"política de DR" ≠ "disaster recovery"	Vetorial — mesmo significado

✅ Quando usar

- Dados **estruturados + textuais** no mesmo pipeline
- Queries que misturam **termos técnicos exatos** com **perguntas abertas**
- Nem vetorial nem keyword sozinhas são suficientes

Latência	300–1000ms
Acurácia	~88%
Custo	$$$
Implementação	2–3 meses

DECISÃO

Matriz de Decisão: Qual RAG Usar?

Cenário	Requisito principal	Tipo recomendado	Justificativa
FAQ e catálogo de serviços	Velocidade, simplicidade	**Basic RAG**	Volume pequeno, queries diretas
Documentação técnica grande	Precisão em volume alto	**Advanced RAG**	Reranking melhora acurácia em escala
ADRs e decisões interconectadas	Rastreabilidade, impacto	**Graph RAG**	Navega relacionamentos e dependências
Múltiplas fontes (wiki + DB + APIs)	Flexibilidade de roteamento	**Agentic RAG**	Agente decide onde buscar
Docs + dados estruturados	Cobertura completa	**Hybrid RAG**	Combina semântica com busca exata

Aspecto	Basic	Advanced	Graph	Agentic	Hybrid
Latência	<100ms	100–500ms	200–800ms	500–2000ms	300–1000ms
Custo	$	$$	$$$	$$$$	$$$
Acurácia	~70%	~85%	~90%	~80%	~88%
Implementação	1–2 sem	3–6 sem	2–4 meses	3–6 meses	2–3 meses

> *"Comece pelo Basic RAG. Meça as métricas. Se precision estiver baixa, adicione reranking. Se o problema for relacionamentos, migre para Graph. Se tiver múltiplas fontes, considere Agentic. Se tiver dados mistos, Hybrid."*

TRADE-OFFS

Trade-offs: Custo × Latência × Acurácia × Governança

Dimensão	O que considerar
Custo	Infraestrutura (Vector DB, Graph DB), modelos de embedding, tokens LLM, equipe de manutenção
Latência	SLA da aplicação, experiência do usuário, batch vs real-time
Acurácia	Criticidade da decisão, consequência de erro, tolerância a alucinação
Governança	Auditabilidade, rastreabilidade de fontes, compliance, multi-tenancy

Cenário 1 — Chatbot de Suporte Interno

Tolerância a erro: média · Latência: <2s · Budget: limitado

→ **Basic RAG ou Advanced RAG**

Cenário 2 — Assistente de Decisão Arquitetural

Tolerância a erro: baixa · Latência: até 10s · Budget: disponível

→ **Graph RAG ou Hybrid RAG**

Cenário 3 — Análise de Compliance Regulatório

Tolerância a erro: zero · Auditabilidade: obrigatória · Múltiplas fontes

→ **Agentic RAG com Hybrid** + human-in-the-loop obrigatório

APLICAÇÕES

RAG para Arquitetos: 5 Aplicações Corporativas

Aplicação	Tipo de RAG	Exemplo
Análise de ADRs históricos	Graph RAG	"Qual a cadeia de decisões que levou à adoção de Kafka?" — navega ADRs interconectados
Consulta de padrões	Basic/Advanced RAG	"Qual o padrão aprovado para comunicação entre microsserviços?" — busca direta
Avaliação de impacto	Graph RAG	"Se descomissionarmos o sistema X, o que é afetado?" — navega dependências
Investigação de incidentes	Agentic RAG	"O que aconteceu na última interrupção do serviço Y?" — logs, tickets, runbooks
Governança de arquitetura	Hybrid RAG	"Estamos em conformidade com a política de segurança v3?" — texto + dados estruturados

O arquiteto como designer de RAG

1. **Identificar o cenário**
   Qual problema estou resolvendo?
2. **Selecionar o tipo**
   Usar a matriz de decisão
3. **Definir trade-offs**
   Custo, latência, acurácia, governança
4. **Especificar componentes**
   Chunking, embedding, vector DB, retrieval
5. **Definir métricas**
   Como medir sucesso?

EXERCÍCIO EM GRUPO

Seleção de RAG para Cenário Corporativo

🏢 Cenário

Consultoria de TI com 15 anos de operação:

- **85 ADRs** interconectados
- **12 wikis** de projetos ativos
- **3 anos** de relatórios de incidentes
- **40 documentos** de segurança e compliance
- **Dados estruturados:** catálogo de sistemas, SLAs, responsáveis
- **3 tipos de usuários:** arquitetos, devs, gestores

⏱️ **Breakout rooms — ~15 minutos**Grupos de 6–7 pessoas

❓ Perguntas do Exercício

1. **Qual tipo de RAG** você escolheria? Justifique com a matriz de decisão.
2. **Qual seria o segundo tipo** mais adequado — e por que foi descartado como primeira opção?
3. **Quais trade-offs práticos** (custo, latência, governança) mais pesariam?
4. **Qual estratégia de chunking** para os ADRs interconectados?
5. **Qual Vector DB** você recomendaria e por quê?

Budget médio, equipe técnica de 3 pessoas. A mesma solução precisa atender os 3 tipos de usuários.

CONSOLIDAÇÃO

Consolidação do Exercício

📊 Poll rápido — "Qual RAG seu grupo escolheu?"

- Basic RAG
- Advanced RAG
- Graph RAG
- Agentic RAG
- Hybrid RAG

🗣️ Discussão dirigida

1. Mostra resultados — convergências e divergências
2. Pede justificativa de 2–3 grupos
3. Cruza com análise do professor

✅ Análise esperada do cenário

1ª Escolha: Graph RAG

ADRs interconectados, análise de impacto, relações complexas. Neo4j como Vector DB natural.

2ª Escolha forte: Hybrid RAG

Dados estruturados (catálogo, SLAs) + textuais (ADRs, wikis). Cobriria os 3 tipos de usuários.

3ª Opção pragmática: Advanced RAG

Se a equipe de 3 pessoas não tiver capacidade para Graph/Hybrid — Advanced com reranking.

Chunking: Document-aware

Preservar seções (Decisão, Justificativa, Consequências) com overlap de 15%.

TAKEAWAYS

Takeaways e Preview da Aula 03

1 **RAG = Grounding em escala**

Resolve knowledge cutoff e alucinação conectando LLMs a dados corporativos

2 **Pipeline tem 4 decisões**

Chunking, embeddings, vector DB, retrieval — cada uma impacta qualidade

3 **5 tipos de RAG para cenários diferentes**

A escolha é uma decisão arquitetural fundamentada em trade-offs reais

4 **Métricas são essenciais**

Precision, recall, MRR, faithfulness — RAG precisa de monitoramento como qualquer sistema

5 **O arquiteto decide, não implementa**

Selecionar tipo, definir trade-offs, especificar componentes

➡️ Preview Aula 03 — Claude Code, MCP e Orquestração

Claude Code avançado · Servidores MCP · Orquestração com Make/n8n · Workflows com human-in-the-loop

> *"Hoje vimos como buscar conhecimento com RAG. Na Aula 03, vamos ver como agir sobre esse conhecimento — conectando agentes a sistemas reais."*

FIAP

MBA